

# How can we explain natural order through the example of Langton's Ant – or how does order arise from disorder?

TIPE report

Hélie EYSSAUTIER-BAVAY –  
49447 2025–2026

## Table of contents

|       |                                                                             |   |
|-------|-----------------------------------------------------------------------------|---|
| 1     | Introduction to Langton's Ant .....                                         | 2 |
| 1.1   | Historical context .....                                                    | 2 |
| 1.2   | Description of the automaton .....                                          | 2 |
| 1.3   | Implementation and observations.....                                        | 3 |
| 2     | Study of the Ant's path.....                                                | 3 |
| 2.1   | Unbounded nature of the path .....                                          | 3 |
| 2.2   | Attractor.....                                                              | 4 |
| 3     | Complexity study of a problem associated with the trajectory .....          | 5 |
| 3.1   | The Ant is capable of universal computation.....                            | 5 |
| 3.1.1 | Control patterns .....                                                      | 5 |
| 3.1.2 | Construction of logic gates .....                                           | 6 |
| 3.1.3 | Conclusion .....                                                            | 6 |
| 3.2   | Complexity of the problem "Will the Ant pass through a given square?" ..... | 6 |
| 4     | Applications in cryptography .....                                          | 7 |
| 4.1   | Generation of pseudo-random bit sequences .....                             | 7 |
| 4.2   | Implementation of a text encryption algorithm.....                          | 8 |
| 4.2.1 | Encryption .....                                                            | 8 |
| 4.2.2 | Decryption .....                                                            | 8 |
| 4.2.3 | Complexity and limitations .....                                            | 9 |
| 5     | Conclusion and scientific openness .....                                    | 9 |

# 1 Introduction to Langton's Ant

## 1.1 Historical context

First appearing in 1940 as a means of modelling self-replicating systems, it was in 1969 that the foundations were laid for a mathematical study of cellular automata as dynamic systems. The computer community of the late twentieth century then resumed this study through various automata (including the famous "Game of Life", conceived by John Conway). In 1986, Christopher Langton, an English biologist, published an article introducing a cellular automaton, known today as "Langton's Ant". Through this automaton, he wanted to illustrate the emergent behaviours observable in nature, showing how complex systems could emerge from cyclical behaviours.

## 1.2 Description of the automaton

Definitions:

- A "Square" is a pair of relative integers  $(i, j)$ .
- "Colouring" is a function  $c: \mathbf{Z} \times \mathbf{Z} \rightarrow \{0, 1\}$ , which associates a square with its colour (a square such that  $c(i, j) = 1$  will be called a black square, and a white square in the opposite case).
- For  $c$ , a colouring, the "Grid associated with  $c$ " is the set of triplets  $(i, j, c(i, j))$  for all integers  $i$  and  $j$ .
- A "finite pattern" is an alternation of black and white squares on a grid, such that there exists  $(i_0, j_0)$  such that: for any  $i$  satisfying  $|i| > i_0$ , for any integer  $j$ ,  $c(i, j) = 0$ ; and for any  $j$  satisfying  $|j| > j_0$ , for any integer  $i$ ,  $c(i, j) = 0$ .
- Otherwise, the grid is called an "infinite pattern".
- A grid composed only of white squares is called a "blank grid".

Definition (cellular automaton): A cellular automaton is a set of squares and an associated colouring, which changes in a discrete manner according to a fixed local rule.

Definition: The Langton Ant is a cellular automaton that works as follows: An Ant is placed on a grid and follows the following rules: if the square on which the Ant is located is white, then the Ant rotates  $90^\circ$  to the left, changes the colour of its square and moves forward one square; if the square is black, the Ant rotates  $90^\circ$  to the right, changes the colour of its square and moves forward one square.

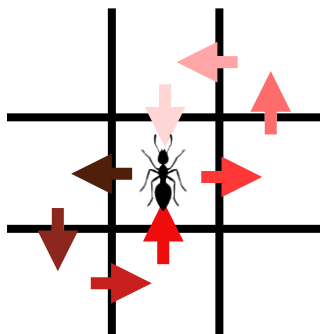


Fig. 1: Diagram of the first steps of execution on the blank grid  
(from the darkest to the lightest arrow)

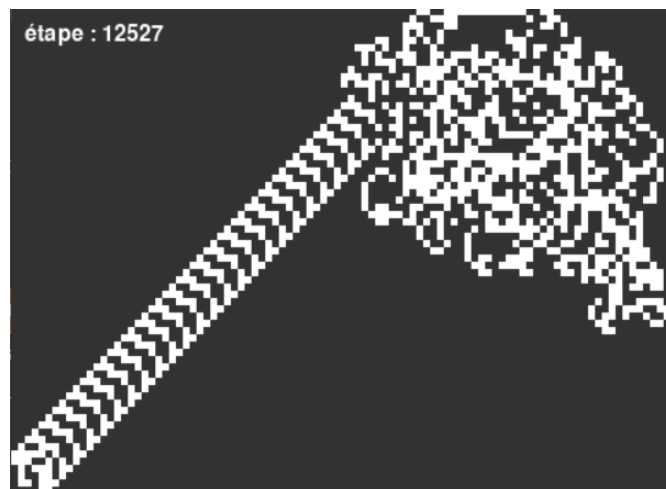
### 1.3 Implementation and observations

Implementation: Experimentally, we can implement the Ant by defining an object "Ant" which contains three functions:

- an initialisation function, which creates the grid (a list of lists of 0 or 1), and initialises the position and direction of the Ant
- a movement function, which modifies the grid, the position and the direction of the Ant according to the rules set out above
- a drawing function, which converts the resulting grid into a screen of black and white squares and displays it.

The code corresponding to this implementation (carried out in Python with the PyGame module) can be found in Appendix 1.

Observations (on the blank grid): The Ant begins by tracing a seemingly chaotic pattern with no observable cycles. After about 10,200 steps, it enters a periodic phase of its movement, tracing the same 104-step pattern indefinitely. The succession of these patterns forms what we will later call "the highway".



*Fig. 2: Appearance of the highway (at ~10,200 steps)*

## 2 Study of the Ant's trajectory

### 2.1 Unbounded nature of the trajectory

*Proposition:*

Whatever the initial grid, the Ant's movement is unbounded.

*Demonstration:*

- 1) As the Ant turns a quarter turn at each step, the grid forms (from the Ant's point of view) a checkerboard of "horizontal" squares that the Ant visits when entering from the East or the West, and "vertical" squares that it visits when entering from the South or the North.
- 2) We reason by contradiction and assume that the Ant has a bounded trajectory:
  - So, since the algorithm dictating the Ant's movement is repeated an infinite number of times, there is a set of squares visited an infinite number of times.
  - Among them, there is one such that the squares to the north and east are not visited an infinite number of times, otherwise the movement would not be bounded.

However, each time the Ant passes over this square, the square changes

colour. We proceed by disjunction of cases:

- a) If the square is a "horizontal" square: The Ant then enters it from the west (the square to the east not being visited an infinite number of times) and exits half the time from the south, and half the time from the north (because the square changes colour each time it is passed over). Thus, the square to the north is visited an infinite number of times, which is absurd.
  - b) If the square is a "vertical" square: The Ant then enters it from the south (the square to the north not being visited an infinite number of times) and exits half the time from the west and half the time from the east (because the square changes colour each time it is passed over). Thus, the square to the east is visited an infinite number of times, which is absurd.
- 3) In all cases, we obtain an absurdity. Thus, the Ant's path is unbounded.

## 2.2 Attractor

*Definition:*

The attractor of a dynamic system is a set of states towards which a system evolves irreversibly in the absence of disturbances.

*Example:*

Thus, the 104-step pattern constituting the highway is an attractor of the Langton Ant.

*Proposition:*

The Langton Ant attractor appears for any initial grid that is a finite pattern. However, this is not true in the case of an infinite pattern.

*Idea for the demonstration:*

- Physical analogy: we equate the Ant with a particle moving in a disordered medium (modelling the Ant as a Lorentz gas: the Ant's movement rules correspond to obstacles that modify the particle's trajectory)
- As the movement is correlated with the state of the grid, there is no diffusion (the Ant does not follow a random movement)
- There are closed orbits, i.e. cycles of movement (the Ant necessarily returns to a configuration it has already visited, apart from translation)
- Thus, any initial configuration converges towards a periodic cycle, and the highway is the only one observed in practice.

(source [7])

Note: This can be observed computationally by implementing an algorithm that runs the Ant on a randomly generated finite pattern (see below). [Annex 2]



Fig. 3: Appearance of the attractor on a finite pattern

### 3 Complexity study of a problem associated with the trajectory

Studying the Ant's trajectory amounts to solving the problem "Will the Ant pass through a given square?" ». We will focus on the complexity of this problem, that is, the time taken to solve this problem algorithmically.

#### 3.1 The Ant is able to evaluate any Boolean circuit

##### 3.1.1 Control patterns

Since the Ant's movement is entirely determined by the grid, we can construct patterns to control its trajectory:

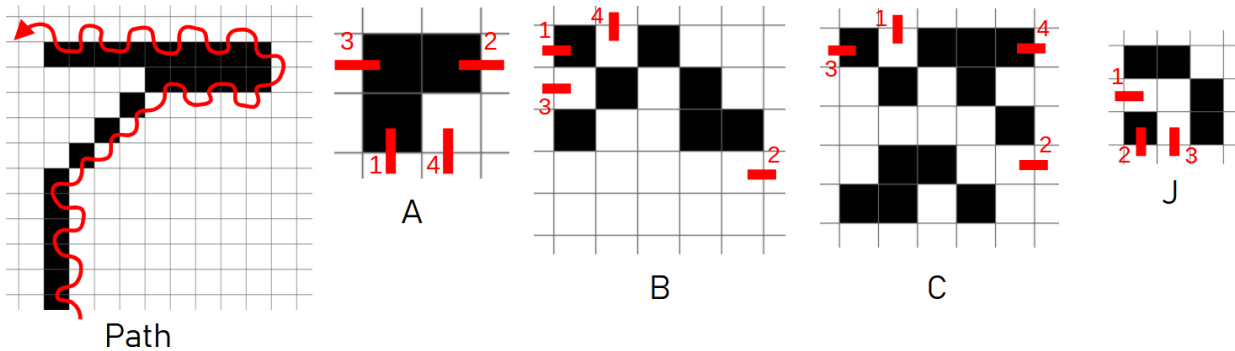


Fig. 4: Patterns for controlling the Ant's path

Here, the "Path" pattern is a path, i.e. it allows the Ant to follow a determined trajectory (according to the red arrow).

Patterns A, B and C are crossing patterns: In pattern A, if the Ant enters through 1, it will exit through 2, and if it then enters through 3, it will exit through 4. In patterns B and C, if the Ant enters through 1, it will exit through 2, and if it enters through 3, it will exit through 4, even if it has not already crossed the pattern to go from 1 to 2.

Pattern J is a junction, that is to say it allows two paths to join: if an Ant enters through 1 or 2, it will exit through 3.

These patterns were taken from the article in the source [2].

### 3.1.2 Construction of logic gates

From these path control patterns, we can derive more complex patterns that will act as logic gates for the Ant:

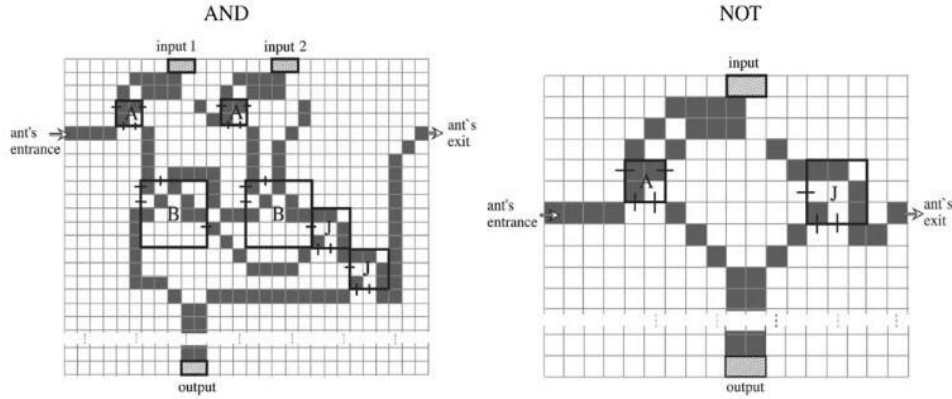


Fig. 4: Patterns representing the AND and NOT logic gates

(images taken from source [2])

By controlling the colours of the input squares, i.e. by using a white square to designate the Boolean *False* and a black square to designate the Boolean *True*, we can, after the Ant has passed through these patterns, observe the colour of the output squares to obtain the evaluation of the AND and NOT logic operations (respectively by the AND and NOT patterns).

### 3.1.3 Conclusion

Thus, the Ant is able to perform any logical operation consisting of the AND and NOT operators. However, the (AND, NOT) system is a complete system of first-order logic, i.e. any logical operation can be rewritten as an operation consisting exclusively of the AND and NOT operators. Thus the Ant is able to perform any logical operation, that is to say, to evaluate any Boolean circuit.

## 3.2 Complexity of the problem "Will the Ant pass through a given square? »

*Definition:*

- A problem is said to be of class P if it is solvable in polynomial time (i.e. it is of polynomial time complexity).
- A problem is said to be P-difficult if any class P problem can be reduced to it in logarithmic space.
- A problem is said to be P-complete if it is P-hard and it is class P

*Examples:*

The problems of finding the shortest path in a graph and finding the maximum in a list are class P problems

*Proposition:*

The problem "Will the Ant pass through a given square?" is P-hard

*Demonstration:*

The problem of evaluating a Boolean circuit (i.e. a logical operation) is a P-complete problem.

However, by solving the problem "Will the Ant pass through a given square?", we solve the problem "Will the Ant pass through the exit square of a given sequence of logic gates? that is to say, the problem of evaluating the corresponding logical operation.

Thus, the problem "Will the Ant pass through a given square?" is at least as difficult to solve as the problem of evaluating a Boolean circuit, i.e. P-hard.

This therefore gives us a reduction in the complexity of this problem. There is currently a debate in the scientific community as to whether this problem is in class P or not, and if not, whether it is in class NP or not.

## 4 Applications in cryptography

### 4.1 Generation of pseudo-random bit sequences

In cryptography, it can be useful to generate pseudo-random sequences, i.e. sequences for which it is very difficult to find an explicit expression, but which can be easily reconstructed with the right initial conditions.

Using Langton's Ant, one way to do this may be to retrieve the colour of the Ant's square at each step, and add a 0 to the sequence if the square is white, and a 1 if the square is black.

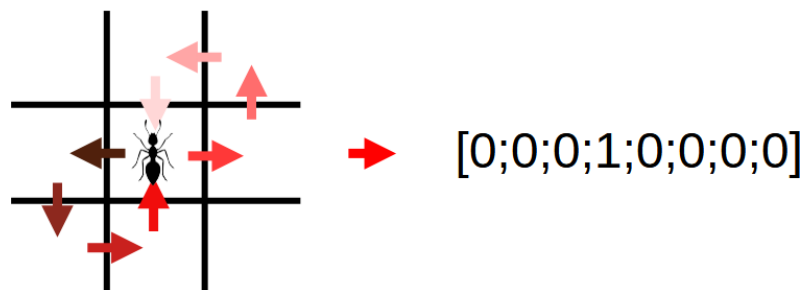
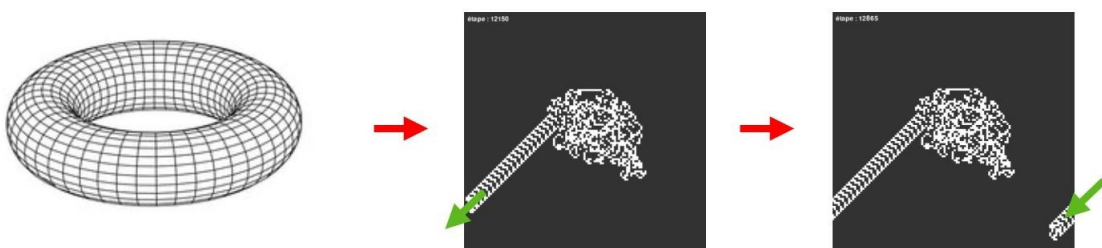


Fig. 5: Generation of a sequence of size 8 corresponding to the ant's first 8 steps on the blank grid

However, we want to avoid the appearance of the attractor, which would make the sequence periodic; one way of doing this may be to place it on a torus, i.e. a finite set of juxtaposed squares, such that the north and south, as well as the east and west, of the screen are connected, so that if the Ant leaves this set of squares from the west, it will return from the east, and vice versa; and if it leaves from the north, it will return from the south, and vice versa.



*Fig. 6: Operation of the ant on a torus*

We can thus artificially limit the Ant's movement by making it move on a screen of finite size, which allows us to generate bit sequences of any size, without the attractor appearing (the Ant will indeed return to its own pattern, which will stop the attractor if it had appeared).

This is used, for example, by the scientists Hosseini et al. ([6]) by combining a cellular automaton with several Ants disrupting the functioning of the former.

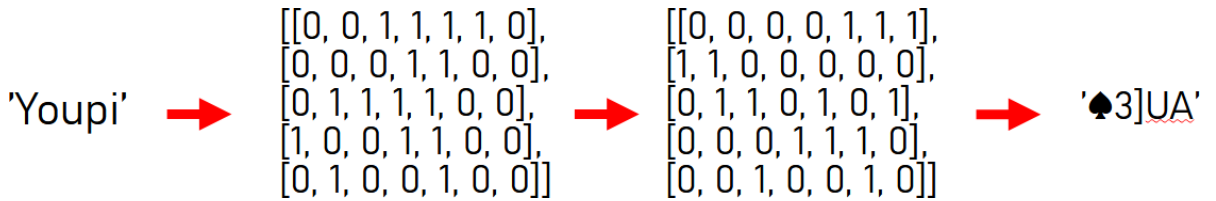
An example of a program for generating pseudo-random bit sequences using the Ant can be found in Annex 3.

## 4.2 Implementation of a text encryption algorithm

### 4.2.1 Encryption

In this section, we propose to implement an algorithm for encrypting character strings using Langton's Ant. To do this, we can convert a given input character string into a torus on which to move the Ant, then convert the resulting torus into a new character string. We can proceed as follows:

- 1) We associate each character with an integer, to which we associate its binary representation, coded on 7 bits (which gives us an alphabet of 128 characters), and represented by a list of 0s and 1s
- 2) The character string is thus transformed into a list of binary representations associated with each character
- 3) We move the Ant on this screen for a number of steps  $n$ , and from an initial position  $(x,y)$ . The triplet  $(n,x,y)$  then constitutes the encryption key
- 4) The resulting screen is transformed into a string of characters by finding the character associated with each line of the final screen.



*Fig. 7: Encryption scheme for the word "Youpi"*

### 4.2.2 Decryption

After encrypting a string of characters, its decryption is made possible by the reversibility of Langton's Ant. Indeed, we notice that after having run the Ant on a screen, if we make it make a U-turn, that is to say a  $180^\circ$  rotation, it retraces its steps, deconstructing its steps as it goes (this is due to the rules it follows).

Thus, the decryption algorithm consists of re-applying the same algorithm as before for a number of steps  $n$ , and from an initial position which is equivalent to the final position  $(x_f, y_f)$



of the Ant during encryption, with an initial direction opposite to the final direction of the Ant during encryption.

The quadruplet  $(n, x_f, y_f, d)$  is thus the decryption key.

The programs corresponding to the encryption and decryption algorithms can be found in Annexes 4 and 5 respectively.

#### *4.2.3 Complexity and limitations*

This encryption algorithm has a complexity of  $O(\max(l, n))$  where  $l$  is the length of the character string, which in reality most often translates to  $O(n)$ ; it therefore offers a relatively low complexity.

However, it can be seen that for long strings of characters, the Ant requires a huge number of steps to go through all the lines and modify all the characters, or at least a significant number of them.

For example, when running the algorithm on 1 million characters, after 100 million steps, we observe that fewer than a thousand characters have been modified, which is therefore very inefficient.

## **5 Conclusion and scientific openness**

Thus, Langton's Ant is a good example of self-organisation and the emergence of order, showing how simple rules can lead to complex behaviours, which leads, in the philosophy of science, to the question: "How can deterministic rules generate chaos? ».

Currently, many points are still under debate in the scientific community, particularly regarding the NP-hardness of the problem studied in part 2, and the Turing-completeness of the Ant. The use of Ant colonies (several Ants moving on the same set of squares, sometimes with slight modifications to the initial rules) is also of interest, for example for the creation of heuristics.

## Bibliographie

- [1] Langton, Christopher G. ; “*Studying artificial life with cellular automata*” ; Physica D: Nonlinear Phenomena 22(1-3): 120-149 (1986)
- [2] A. Gajardo, A. Moreira, E. Goles ; “*Complexity of Langton's ant*” ; Discrete Applied Mathematics, Volume 117, Issues 1–3, 2002, Pages 41-50
- [3] D. Gale ; “*The Industrious Ant*” ; Mathematical Intelligencer, vol. 15, no. 2 (1993), p 54–58
- [4] O. Carton ; “*Langages formels, calculabilité et complexité*” ; Vuibert, 2008
- [5] Takeo Hagiwara, Tatsuie Tsukiji ; “*Hardness of Approximation for Langton's Ant on a Twisted Torus*” ; *Algorithms* 13, no. 12: 344. <https://doi.org/10.3390/a13120344> (2020)
- [6] Hosseini S.M. , Karimi H. , Jahan M.V. ; “*From Complexity to Random Behaviors; Generate Random Numbers by Confusion in Cellular Automata State's*” ; Proceedings of the International Conference on Scientific Computing (CSC), Las Vegas, NV, USA, 18–21 juillet 2011, p. 1
- [7] Bunimovich L.A.; Troubetzkoy S.E.; “*Rotators, periodicity, and absence of diffusion in cyclic cellular automata*”; Journal of Statistical Physics, 1994

## Annexes

```
1 import pygame
2 import sys
3
4 pygame.init()
5
6 scr_h=1
7 scr_w=1
8 cell_size=5
9 grid_h=scr_h//cell_size
10 grid_w=scr_w//cell_size
11
12 black=(255,255,255)
13 white=(50,50,50)
14
15 cpt = 0
16
17 Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
18
19 class langton_ant:
20     def __init__(self):
21         self.x=grid_w//2
22         self.y=grid_h//2
23         self.dir=0
24         self.grid=[[0]*grid_w for _ in range (grid_h)]
25
26     def move(self):
27         crt_cell=self.grid[self.y][self.x]
28         if crt_cell==0:
29             self.grid[self.y][self.x]=1
30             self.dir=(self.dir + 1)%4
31         else:
32             self.grid[self.y][self.x]=0
33             self.dir=(self.dir - 1)%4
34
35         self.x+=Dir[self.dir][0]
36         self.y+=Dir[self.dir][1]
37         return (self.x, self.y)
38
39     def draw(self, scr):
40         for y in range (grid_h):
41             for x in range (grid_w):
42                 color = white if self.grid[y][x]==0 else black
43                 pygame.draw.rect(scr, color, (x*cell_size, y*cell_size, cell_size, cell_size))
44
45     def exp_grid_e(self):
46         global grid_h, grid_w, scr_h, scr_w
47         new_grid=[[0]*(grid_w+1) for _ in range(grid_h)]
48
49         for y in range (grid_h):
50             for x in range(grid_w):
51                 new_grid[y][x]=self.grid[y][x]
52
53         self.grid=new_grid
54         grid_w+=1
55         grid_h+=0
56
57         #self.x+=0
58         #self.y+=0
59
60         scr_h+=cell_size*0
61         scr_w+=cell_size*1
62         pygame.display.set_mode((scr_w, scr_h))
63
64     def exp_grid_o(self):
65         global grid_h, grid_w, scr_h, scr_w
66         new_grid=[[0]*(grid_w+1) for _ in range(grid_h)]
67
68         for y in range (grid_h):
69             for x in range(grid_w):
70                 new_grid[y][x+1]=self.grid[y][x]
71
72         self.grid=new_grid
73         grid_w+=1
74         grid_h+=0
75
76         self.x+=1
77         #self.y+=0
```

*Annexe 1 : Code du programme permettant de calculer et afficher la fourmi en configuration initiale*

```

78     scr_h+=cell_size*0
79     scr_w+=cell_size*1
80     pygame.display.set_mode((scr_w, scr_h))
81
82
83     def exp_grid_s(self):
84         global grid_h, grid_w, scr_h, scr_w
85         new_grid=[[0]*(grid_w) for _ in range(grid_h+1)]
86
87         for y in range (grid_h):
88             for x in range(grid_w):
89                 new_grid[y][x]=self.grid[y][x]
90
91         self.grid=new_grid
92         grid_w+=0
93         grid_h+=1
94
95         #self.x+=0
96         #self.y+=0
97
98         scr_h+=cell_size*1
99         scr_w+=cell_size*0
100        pygame.display.set_mode((scr_w, scr_h))
101
102        def exp_grid_n(self):
103            global grid_h, grid_w, scr_h, scr_w
104            new_grid=[[0]*(grid_w) for _ in range(grid_h+1)]
105
106            for y in range (grid_h):
107                for x in range(grid_w):
108                    new_grid[y+1][x]=self.grid[y][x]
109
110            self.grid=new_grid
111            grid_w+=0
112            grid_h+=1
113
114            #self.x+=0
115            self.y+=1
116
117        scr_h+=cell_size*1
118        scr_w+=cell_size*0
119        pygame.display.set_mode((scr_w, scr_h))
120
121    scr = pygame.display.set_mode((scr_w, scr_h))
122    pygame.display.set_caption("Fourmi de Langton (main)")
123
124    font=pygame.font.Font(None, 24)
125
126    ant=langton_ant()
127    clock = pygame.time.Clock()
128
129    a=(grid_w//2, grid_h//2)
130    while 1 :
131        for e in pygame.event.get():
132            if e.type==pygame.QUIT:
133                pygame.quit()
134                sys.exit()
135            if a[0]>=grid_w:
136                ant.exp_grid_e()
137            if a[0]<0:
138                ant.exp_grid_o()
139            if a[1]>=grid_h:
140                ant.exp_grid_s()
141            if a[1]<0:
142                ant.exp_grid_n()
143        a=ant.move()
144        scr.fill(white)
145        ant.draw(scr)
146        cpt+=1
147        txt = font.render("étape : "+str(cpt),1,(255,255,255))
148        scr.blit(txt, (10, 10))
149        pygame.display.flip()
150        #clock.tick(3)
151

```

*Annexe 1 : Code du programme permettant de calculer et afficher la fourmi en configuration initiale*

```

1 import pygame
2 import sys
3 import random
4
5 pygame.init()
6
7 dim_h=random.randint(0,100)
8 dim_w=random.randint(0,100)
9
10 cell_size=5
11 scr_h=dim_h*cell_size
12 scr_w=dim_w*cell_size
13
14 grid_h=dim_h
15 grid_w=dim_w
16
17 black=(255,255,255)
18 white=(50,50,50)
19
20 cpt = 0
21
22 Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
23
24 class langton_ant:
25     def __init__(self):
26         self.x=grid_w//2
27         self.y=grid_h//2
28         self.dir=0
29         self.grid=[[random.randint(0,1) for _ in range (grid_w)] for _ in range (grid_h)]
30

```

*Annexe 2 : Code du programme permettant de calculer et afficher la fourmi avec un motif initial rectangulaire fini généré aléatoirement (la suite du programme est la même que celui de l'annexe 1)*

```

1 import random
2
3 n=int(input("Longueur de la séquence : "))
4 x_init, y_init=tuple(map(int,input("encryption key with the shape x y : ").split()))
5
6
7 scr_h=2400
8 scr_w=2400
9 cell_size=8
10 grid_h=scr_h//cell_size
11 grid_w=scr_w//cell_size
12
13 thorus=[[random.randint(0,1) for _ in range (grid_w) ] for _ in range(grid_h)]
14
15 white=[50,50,50]
16 black=[255,255,255]
17
18 cpt=0
19
20 Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
21
22 bit_list=[]
23
24 class langton_ant:
25     def __init__(self):
26         self.x=x_init%grid_w
27         self.y=y_init%grid_h
28         self.dir=0
29         self.grid=thorus
30
31     def move(self):
32         crt_cell=self.grid[self.y][self.x]
33         if crt_cell==0:
34             self.grid[self.y][self.x]=1
35             self.dir=(self.dir + 1)%4
36             b=0
37         else:
38             self.grid[self.y][self.x]=0
39             self.dir=(self.dir - 1)%4
40             b=1
41

```

*Annexe 3 : Code du programme de génération pseudo-aléatoire de bits*

```

42         self.x+=Dir[self.dir][0]
43         self.y+=Dir[self.dir][1]
44
45         self.x%=grid_w
46         self.y%=grid_h
47         return b
48
49     ant=langton_ant()
50
51     while n>0 :
52         b=ant.move()
53         bit_list.append(b)
54         n-=1
55
56     print("séquence de bits : ", bit_list)
57     print("encryption_key : ", (x_init,y_init))
58

```

*Annexe 3 : Code du programme de génération pseudo-aléatoire de bits*

```

1  mess=str(input("message to encrypt : "))
2  p=tuple(map(int,input("encryption key with the shape x y : ").split()))
3  nb=int(input("number of iterations : "))
4
5  def crypt(grid,p,nb):
6
7      dim_h=len(grid)
8      dim_w=7
9
10     grid_h=dim_h
11     grid_w=dim_w
12
13     cpt = 0
14
15     Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
16
17     class langton_ant:
18         def __init__(self):
19             self.x=p[0]%grid_w
20             self.y=p[1]%grid_h
21             self.dir=0
22             self.grid=grid
23
24         def move(self):
25             crt_cell=self.grid[self.y][self.x]
26             if crt_cell==0:
27                 self.grid[self.y][self.x]=1
28                 self.dir=(self.dir + 1)%4
29             else:
30                 self.grid[self.y][self.x]=0
31                 self.dir=(self.dir - 1)%4
32
33             self.x+=Dir[self.dir][0]
34             self.y+=Dir[self.dir][1]
35
36             self.x%=grid_w
37             self.y%=grid_h
38             return self.grid, (self.x,self.y), self.dir
39

```

*Annexe 4 : code du programme de chiffrement de texte*

```

40 ant=langton_ant()
41 if nb==0:
42     mess2=[[grid[i][j] for j in range(grid_w)] for i in range(grid_h)],p,0)
43     for i in range(nb) :
44         if i==nb-1:
45             mess2=ant.move()
46         else:
47             ant.move()
48         cpt+=1
49     return mess2
50
51 alph=['0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f',
52       'g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z',
53       'A','B','C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z',
54       '[','\','_','{','|','}','~','`','^','>','?','@','#','$','%','&','\'','\"',' ','(',')','(',')','(',')','(',')',
55       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n',
56       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n',
57       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n',
58       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n',
59       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n',
60       '\t','\n','\r','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n','\t','\f','\b','\r','\n']
61
62 def findmax(n):
63     i=0
64     maxi=0
65     while 2**i <= n:
66         maxi=i
67         i+=1
68     return maxi
69
70 def ten2two(n):
71     i=findmax(n)
72     n2=[0 for _ in range(7)]
73     while n!=0:
74         n2[i]=1
75         n-=2**i
76         i=findmax(n)
77     return n2
78
79 def two2ten(n2):
80     n=0
81     for i in range(len(n2)):
82         if n2[i]==1:
83             n+=2**i
84     return n
85
86 def str2seq(S, a):
87     L=[]
88     for s in S:
89         i=0
90         while s!=a[i]:
91             i+=1
92         n2=ten2two(i)
93         L.append(n2)
94     return L
95
96 def seq2str(L,a):
97     S=""
98     for l in L:
99         i=two2ten(l)
100        S+=a[i]
101    return S
102
103 Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
104
105 temp=crypt(str2seq(mess,alpha),p,nb)
106 tempo=temp[0]
107 dim_h=len(tempo)
108 dim_w=7
109
110 grid_h=dim_h
111 grid_w=dim_w
112 d=temp[2]
113 pf=((temp[1][0]-Dir[d][0])%grid_w, (temp[1][1]-Dir[d][1])%grid_h)
114 print(temp)
115 print("crypté message :", seq2str(tempo,alpha))
116 print("decryption key :", pf)
117 print("number of iterations :", nb)
118 print("final direction :", d)

```

```

1  crypted=str(input("message to decrypt : "))
2  decryption_key=tuple(map(int,input("decryption key with the shape x y : ").split()))
3  nb=int(input("number of iterations : "))
4  d=int(input("initial direction : "))
5
6  def decrypt(grid,p,nb,d):
7
8      dim_h=len(grid)
9      dim_w=7
10
11      grid_h=dim_h
12      grid_w=dim_w
13
14      cpt = 0
15
16      Dir=[(0,-1), (1,0), (0,1), (-1, 0)]
17
18      class langton_ant:
19          def __init__(self):
20              self.x=p[0]
21              self.y=p[1]
22              self.dir=(d+2)%4
23              self.grid=grid
24
25          def move(self):
26              crt_cell=self.grid[self.y][self.x]
27              if crt_cell==0:
28                  self.grid[self.y][self.x]=1
29                  self.dir=(self.dir + 1)%4
30              else:
31                  self.grid[self.y][self.x]=0
32                  self.dir=(self.dir - 1)%4
33
34              self.x+=Dir[self.dir][0]
35              self.y+=Dir[self.dir][1]
36
37              self.x%=grid_w
38              self.y%=grid_h
39              return self.grid
40
41      ant=langton_ant()
42      if nb==0:
43          mess2=[[grid[i][j] for j in range(grid_w)] for i in range(grid_h)]
44      for i in range(nb) :
45          if i==nb-1:
46              mess2=ant.move()
47          else:
48              ant.move()
49          cpt+=1
50      return mess2
51
52      alph=['0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f',
53          'g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z','A','B','C','D','E','F','G','H','I',
54          'J','K','L','M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z','!','@','#','$','%','&','(',')','*','+','-','_','`',
55          '~','{','}','|','^','\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
56          '\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
57          '\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
58          '\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
59          '\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
60          '\n','\t','\r','\f','\b','\a','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u','\e','\c','\b','\a','\u',
61
62      def findmax(n):
63          i=0
64          maxi=0
65          while 2**i <= n:
66              maxi=i
67              i+=1
68          return maxi
69
70      def ten2two(n):
71          i=findmax(n)
72          n2=[0 for _ in range(7)]
73          while n!=0:
74              n2[i]=1
75              n-=2**i
76              i=findmax(n)
77          return n2
78

```

*Annexe 5 : code du programme de déchiffrement de texte*



```

79 def two2ten(n2):
80     n=0
81     for i in range (len(n2)):
82         if n2[i]==1:
83             n+=2**i
84     return n
85
86 def str2seq(S, a):
87     L=[]
88     for s in S:
89         i=0
90         while s!=a[i]:
91             i+=1
92         n2=ten2two(i)
93         L.append(n2)
94     return L
95
96 def seq2str(L,a):
97     S=''
98     for l in L:
99         i=two2ten(l)
100         S+=a[i]
101     return S
102
103 print("decrypted message :", seq2str(decrypt(str2seq(crypted, alph),decryption_key,nb,d),alph))
104

```

*Annexe 5 : code du programme de  
déchiffrement de texte*